

Too Much ACID, Not Enough Reality

A PostgreSQL tour of what your SELECT thinks happened

Vinicius Grippa

Lead Database Engineer

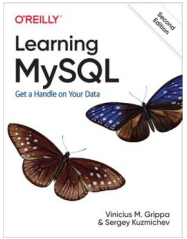
readyset

About Me



SELECT * FROM person WHERE name LIKE 'Vini';

- Lead Database Engineer at **Readyset**
- Previously at Percona, working across database technologies
- Co-author of the book Learning MySQL
- Organizer of the MySQL community in Brazil (MUG)



readyset

Three questions about freshness

01 PostgreSQL How old is the answer when it arrives?

02 Replica How much extra delay does replication add?

03 Readyset What changes when Readyset maintains the result?

PostgreSQL semantics, illustrated timelines, and a published cache benchmark

readyset

What 4-letter word defines how database transactions behave?

readyset

A	**PRAY**	Pray to work. Pay the sacrifice to the gods.
B	**YOLO**	Real devs test in production
C	**F*CK**	Forgot the WHERE clause
D	**ACID**	Atomicity, Consistency, Isolation, Durability *(boring, but correct)*

readyset

ACID

What each letter guarantees in PostgreSQL, and where the guarantee ends

readyset

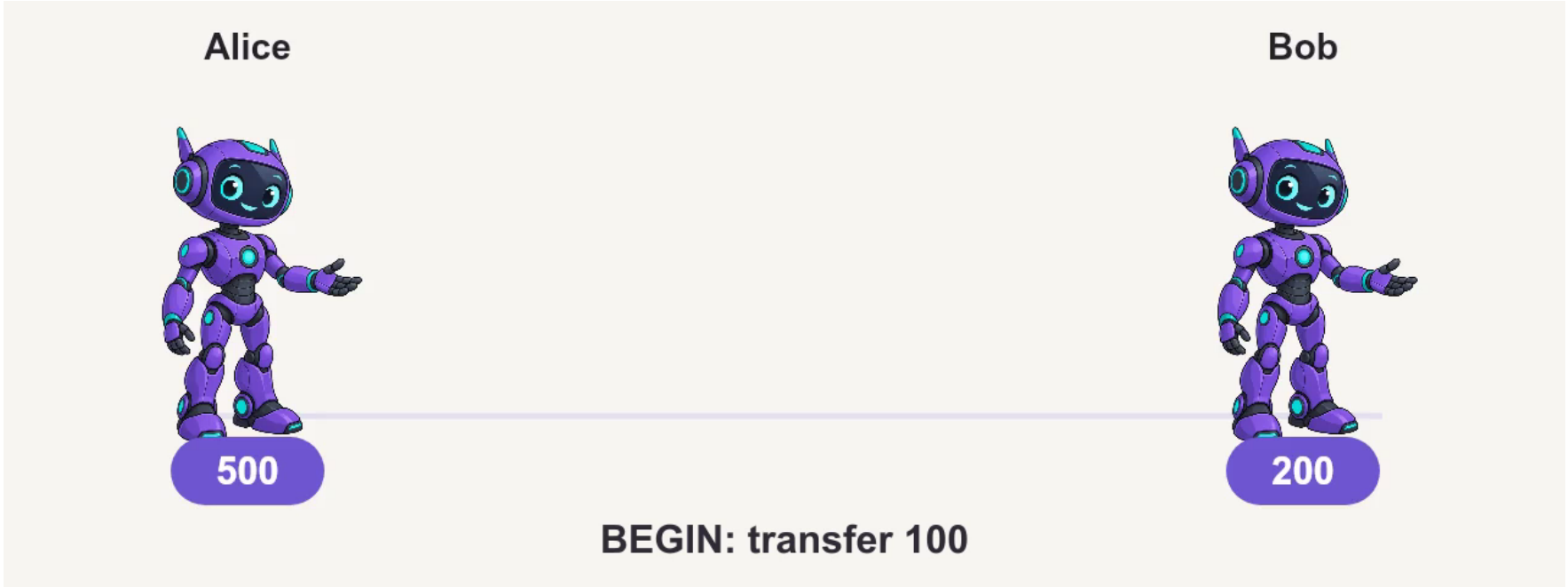
ACID: four promises of a transaction

Example: transfer 100 from Alice to Bob.

A	**Atomicity** All or nothing	Debit and credit are one unit. Commit both, or roll back both.
C	**Consistency** Valid state to valid state	The transaction must obey declared rules and constraints.
I	**Isolation** Concurrent work, controlled	Controls what transactions see and change. Guarantees depend on the isolation level.
D	**Durability** Committed means saved	After a successful commit, changes survive a crash or power loss with durable settings.

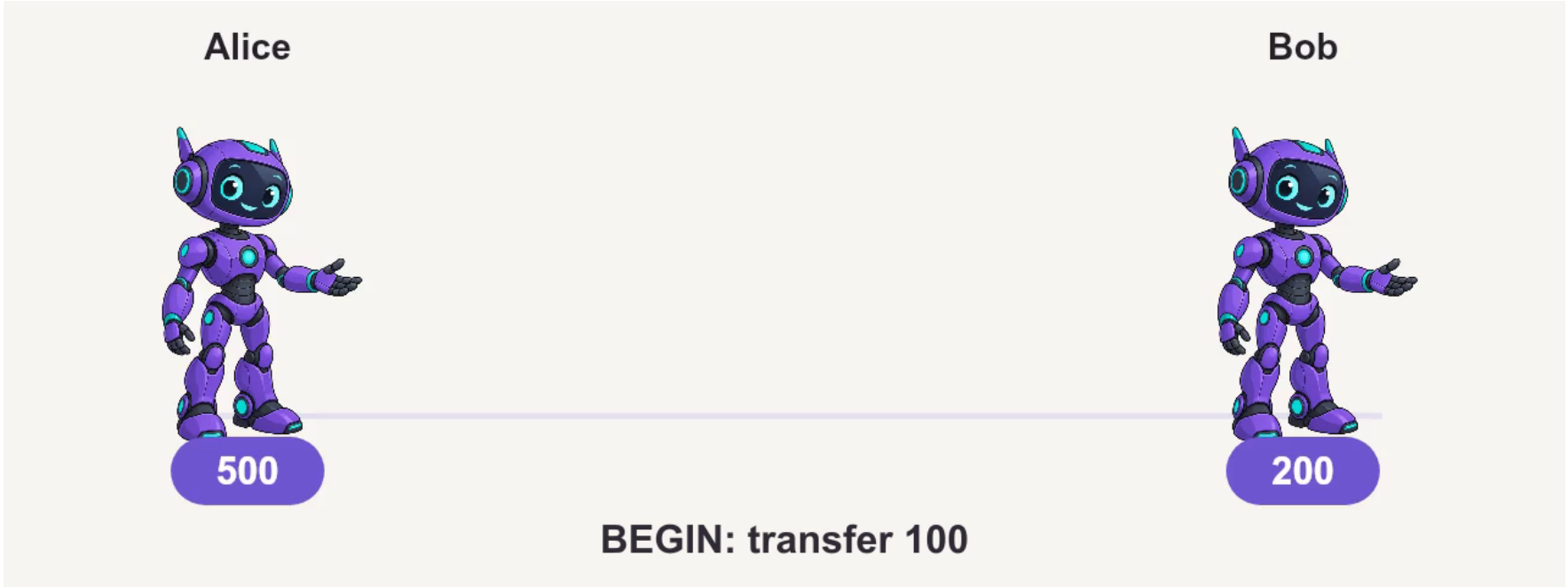
readyset

Atomicity: commit



readyset

Atomicity: rollback



readyset

Consistency



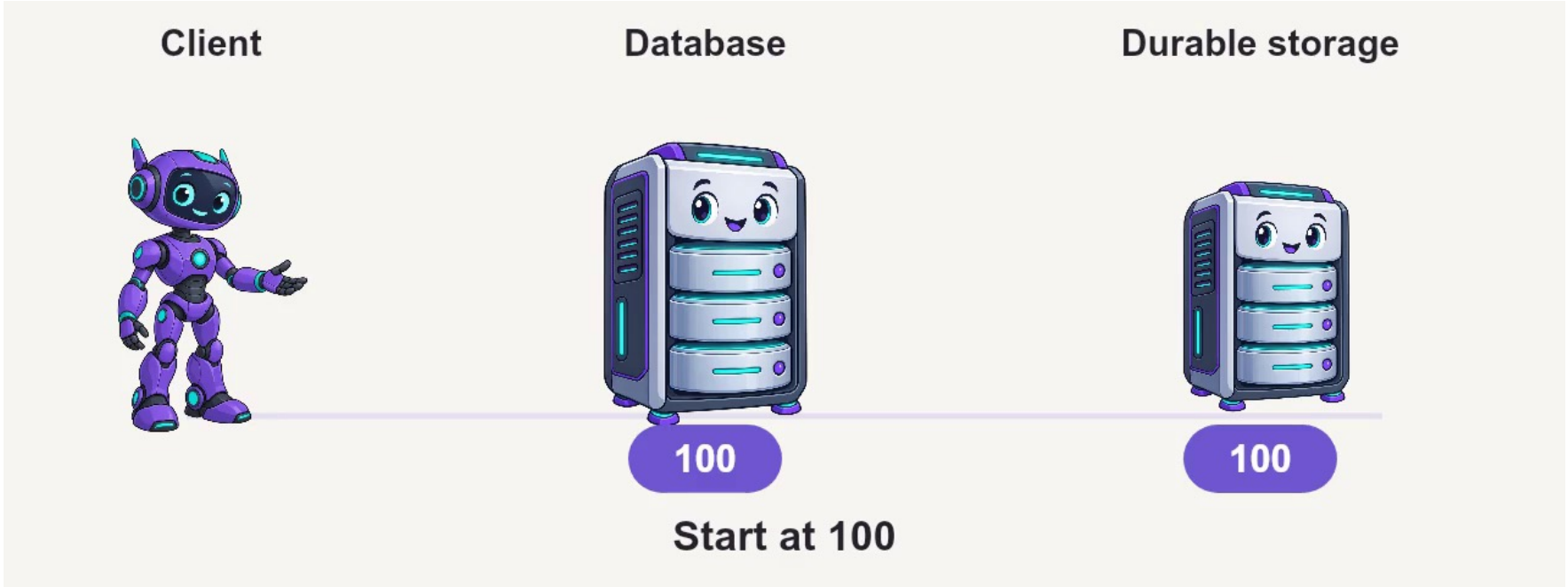
readyset

Isolation



readyset

Durability



readyset

MVCC: tuple versions and VACUUM



ROLLBACK

Aborted changes stay invisible

SNAPSHOT READ

Choose a visible tuple version

Long snapshots delay cleanup. VACUUM reclaims versions no snapshot needs.

readyset

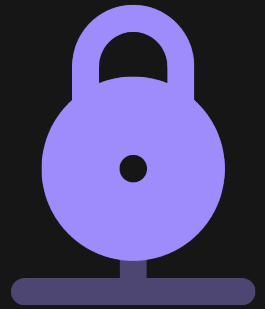
Autocommit still means a transaction

Autocommit	statement 1	COMMIT		
	statement 2	COMMIT		
Explicit transaction	BEGIN	statement 1	statement 2	COMMIT

READ COMMITTED: a snapshot per statement. REPEATABLE READ: one per transaction.

readyset

Isolation levels



Statement snapshots, transaction snapshots, and serialization failures

readyset

Read Uncommitted behaves like Read Committed



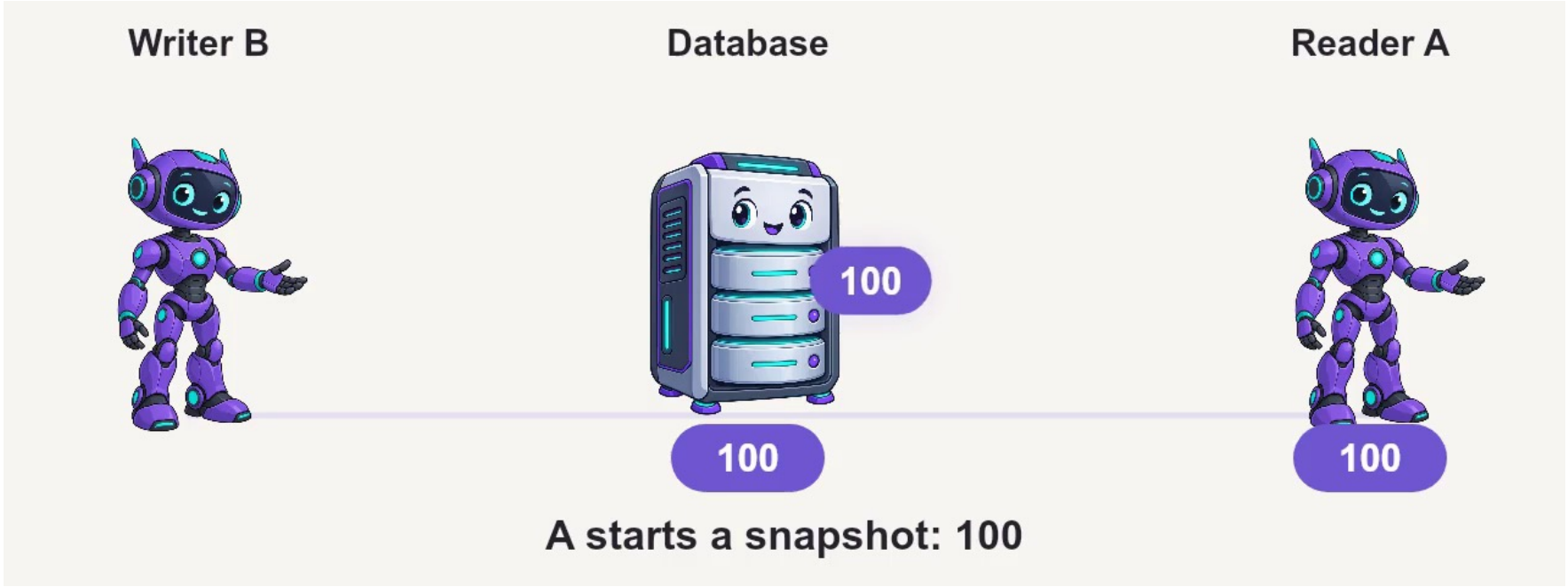
readyset

Read Committed: PostgreSQL's default



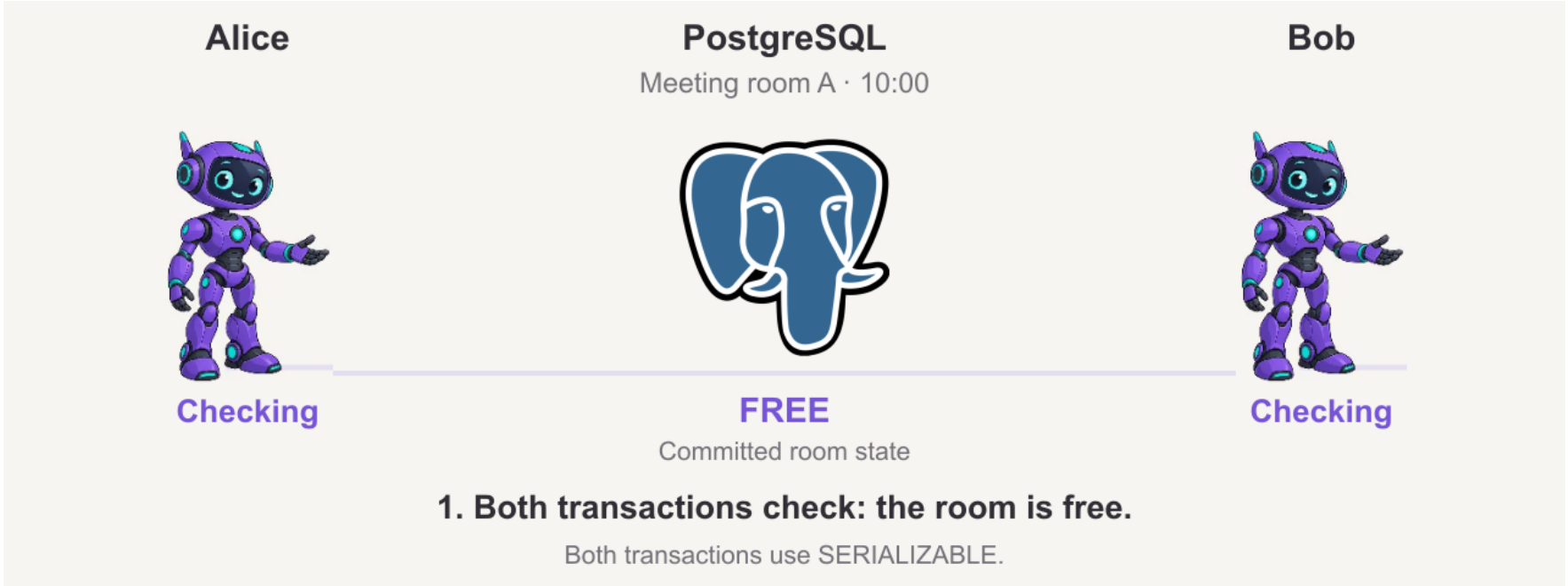
readyset

Repeatable Read: a stable snapshot



readyset

Serializable: two bookings, one room



Local PostgreSQL 18.3 test. Animation timing is illustrative.

readyset

Physical standby: WAL replay and read snapshots

Recovery process

WAL replay

Visibility advances after commit replay

Read-only client

RC or RR snapshot

A long snapshot can conflict with recovery

Replay lag and snapshot age both matter.

Readyset



A decade of research. One problem.

origin

A decade of MIT research behind it

The Noria paper (OSDI '18) describes the **partial dataflow** model Readyset is built on — from MIT PDOS, a systems group focused on parallel and distributed data.

mission

“There are only two hard things in Computer Science: cache invalidation and naming things.”

— Phil Karlton

Readyset solves the first. You're on your own for the second.

two modes

Shallow Caching

TTL-based result cache. Lightweight setup, broad coverage.

Deep Caching

Incremental dataflow via CDC/IVM. Eventual consistency.

paper — OSDI '18

Noria: dynamic, partially stateful data-flow for high performance web applications

Jon Gjengset - MIT PDOS · 2018

pdos.csail.mit.edu/papers/noria:osdi18.pdf

open source

github.com/readysettech/readyset

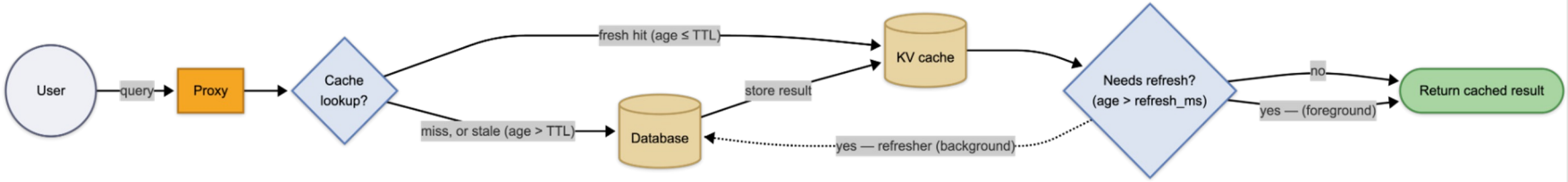
readyset

Shallow Caching



How a query moves through Readyset

TTL-based results with background refresh for eligible entries



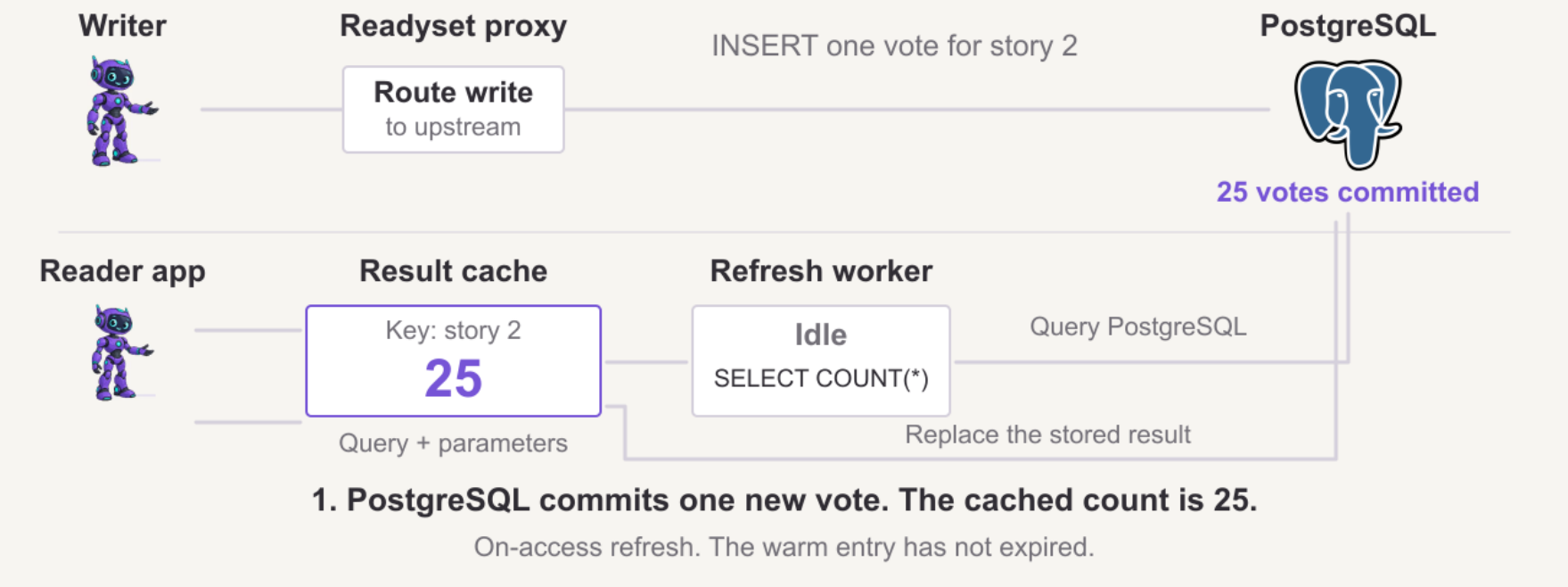
Proxy routes queries. Cache stores results. Misses and refreshes query PostgreSQL.

outcome

A refresh can serve the existing result while a background query runs. Cold or expired entries can wait for upstream.

readyset

Shallow cache: refresh after a write



Warm key: story 2. Conceptual simulation. Timing is illustrative.

Shallow caching in PostgreSQL applications

- **Access control: ordinary cached results are shared**
- **Coalescing: concurrent misses can share upstream work**
- **TTL: entry lifetime follows last access**
- **Refresh: eligible entries refresh in the background**
- **RLS: supported policies partition keys by session state**
- **RLS refresh: in-session on a miss, not in the background**
- **Policy changes: catalog polling adds a staleness window**
- **Deep caching does not support RLS-protected tables**

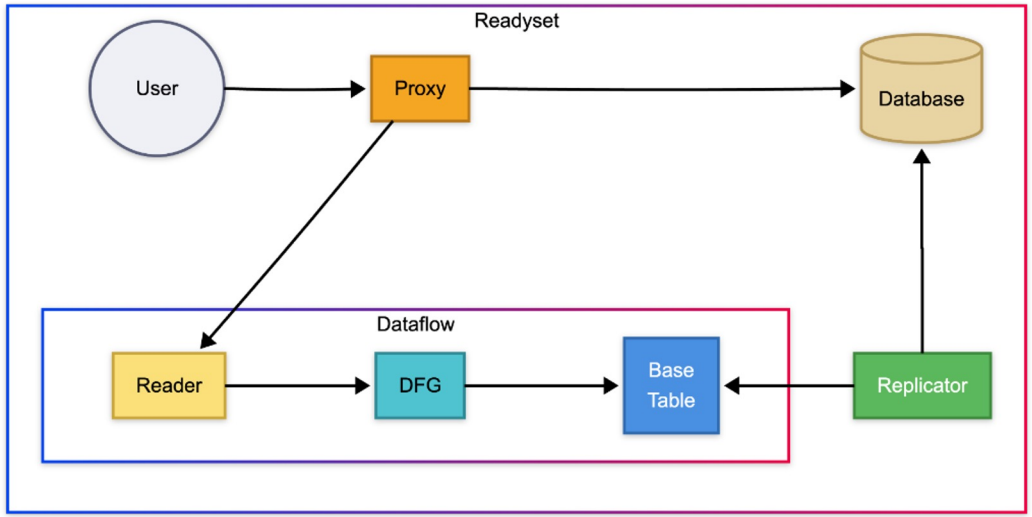
readyset

Deep Caching



The database notifies. Readyset keeps up.

Initial snapshot, logical replication, and incremental maintenance



reads → served by the Reader · writes → flow straight to the Database · Replicator feeds the Dataflow

components

What makes up Deep

01. **Snapshot**
An initial copy of the tables that will be cached — hydrates the Base Tables of the dataflow.

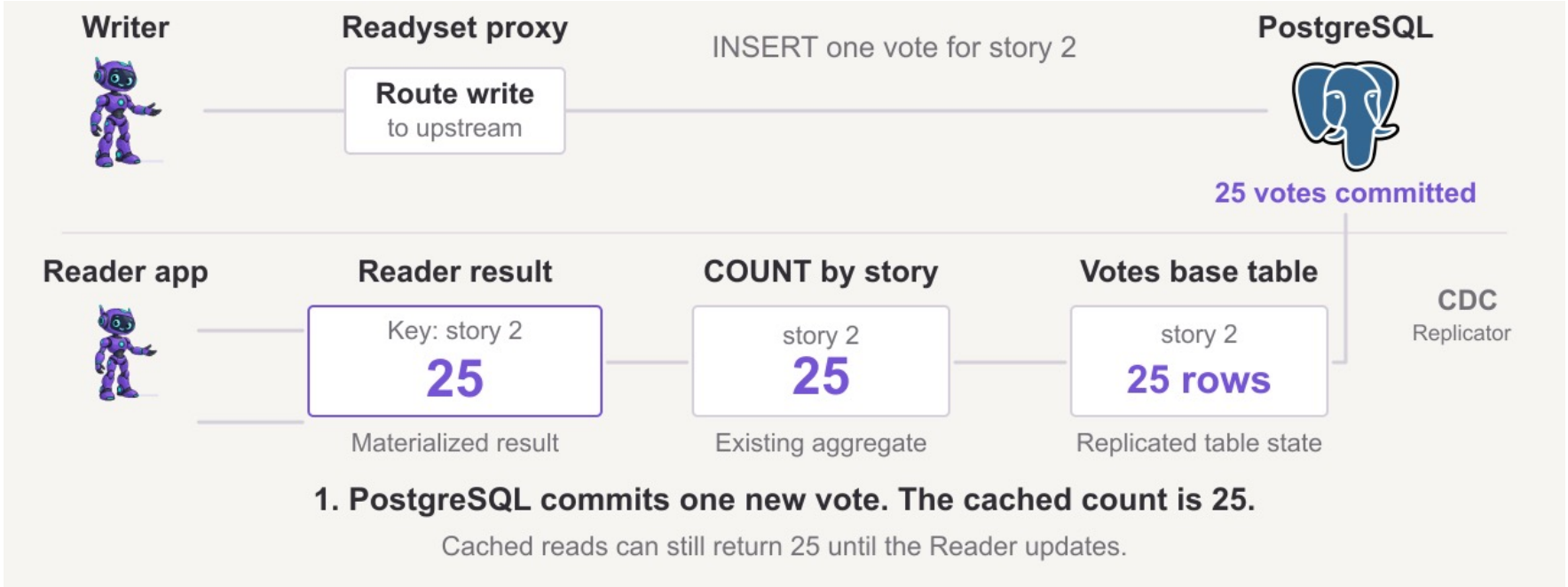
02. **Replication / CDC**
Streams PostgreSQL changes through a logical replication slot.

03. **Proxy**
Speaks the PostgreSQL wire protocol. Routes cached reads or proxies upstream.

04. **Dataflow** // heart of the system
Incremental graph: Base Table → DFG → Reader. It recomputes only what changed. Details in the next slides.

readyset

Deep cache: propagate a write

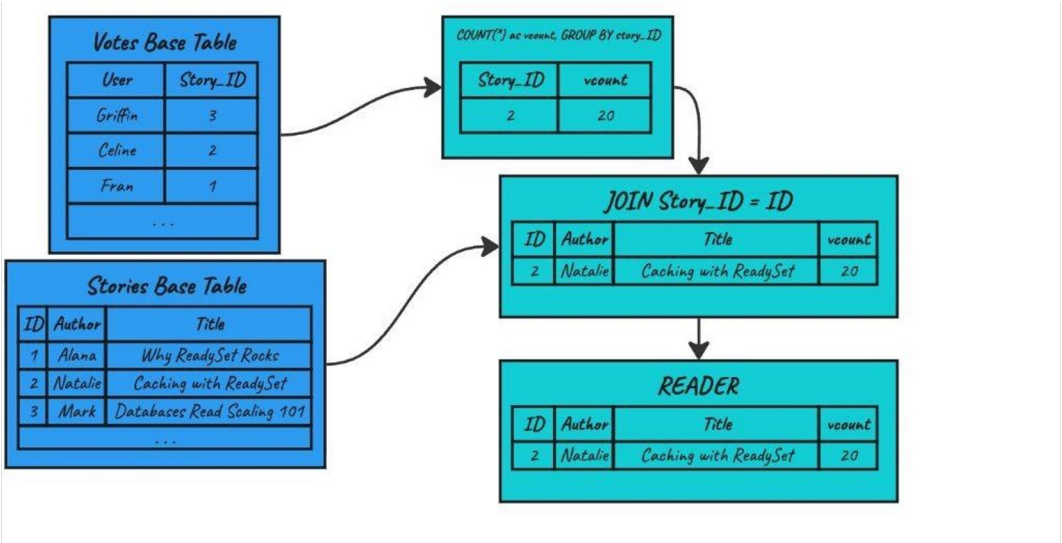


Warm key: story 2. Conceptual simulation. Timing is illustrative.

Supported SQL operators become dataflow nodes

example query

```
SELECT s.id, s.author, s.title, COUNT(*) AS vcount
FROM stories s JOIN votes v ON v.story_id = s.id WHERE s.id = $1 GROUP BY s.id, s.author, s.title;
```

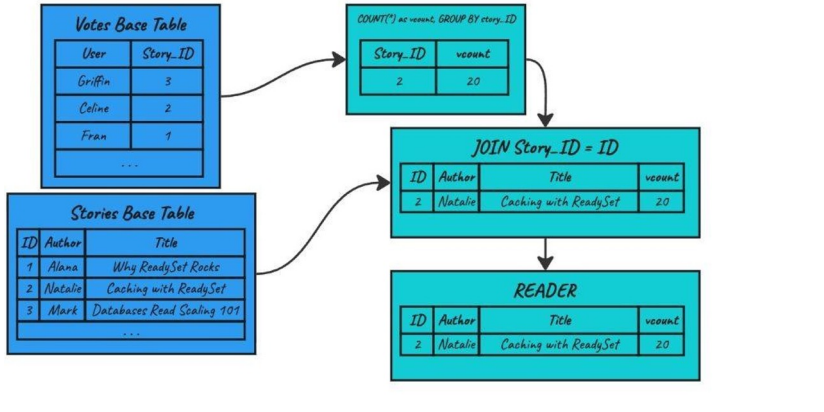


Partial materialization

building the graph

The graph maintains results as data changes.

- **Base Tables.** In-memory copies of PostgreSQL tables — hydrated by the initial Snapshot and updated by the Replicator.
- **Intermediate operators.** The query parser becomes a graph: GROUP BY, JOIN, FILTER, COUNT — each one a node.
- **Reader.** The terminal node — stores the final result set handed to the client, keyed for lookup.
- **Partial materialization. Intermediate results can be demand-driven. Base tables still retain replicated state.**

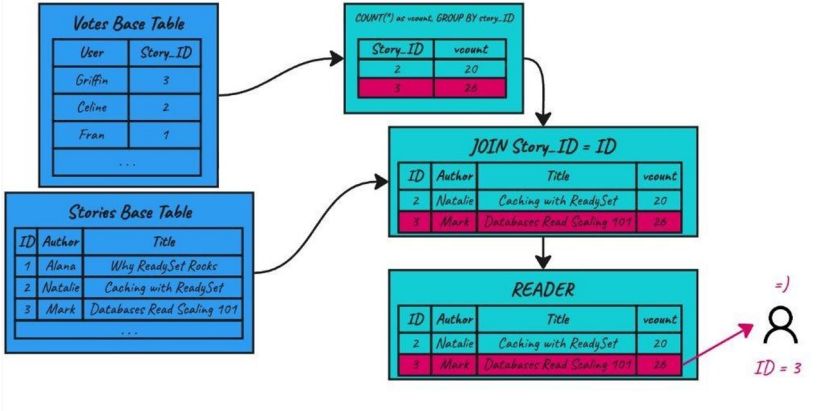


Cache miss — compute on demand

a new key arrives

A user requests id = 3 — but it isn't materialized yet.

- **Miss at the Reader.** The key id=3 isn't available in the materialized result.
- **Upquery.** The Reader fires a reverse query — walks up the graph asking only for the rows relevant to story_id=3.
- **On-demand computation.** Base Tables filter and return only what's needed → the aggregator recomputes vcount=26 → the JOIN completes with the stories row.
- **A new result is materialized. The Reader now holds id=2 and id=3 for subsequent lookups.**
- **Concurrent misses. Upqueries can share work. Cold lookups still pay computation and scheduling costs.**



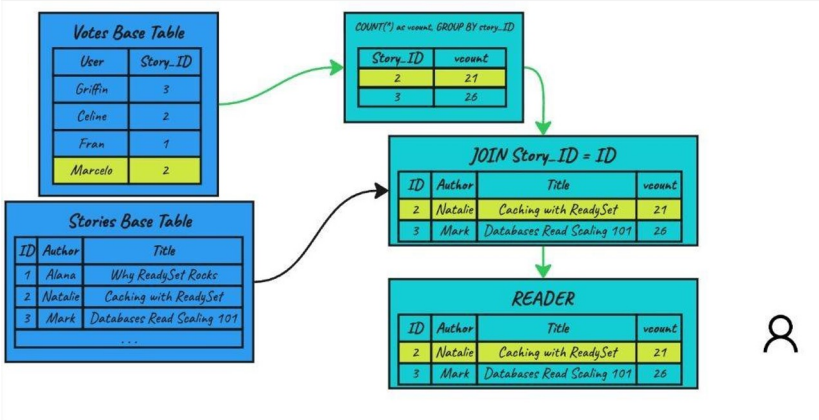
Incremental view maintenance

a write arrives — what now?

A new vote from Marcelo on story 2. The delta propagates.

- **CDC delivers the delta.** The Replicator captures the INSERT on votes and feeds the Base Table.
- **Incremental propagation.** The aggregator receives the delta and increments: vcount 20 → 21. It does not recompute the COUNT over every vote — only the cached value is adjusted.
- **JOIN refreshed.** The new vcount=21 reaches the JOIN node, which replaces the row for id=2.
- **Reader converges. After the delta reaches the Reader, a lookup for id=2 sees vcount=21.**
- **Update cost depends on affected state, operator work, and join fan-out.**

No TTL for deep caches. Freshness still depends on CDC and dataflow lag.



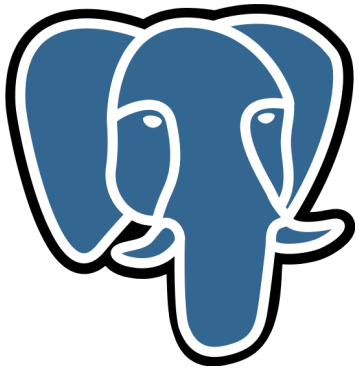
readyset

THE FRESHNESS LAB

“Read from the source. It is always fresh.”

Is it still fresh when the answer arrives?

A running SELECT keeps an earlier snapshot



→

Snapshot at 0 s

The query keeps this view

New commits stay outside it

Source keeps changing

READ COMMITTED too: an ordinary SELECT keeps its statement snapshot while writes continue

readyset

10 seconds. 30 seconds. One minute.

Snapshot at 0 s → answer arrives later

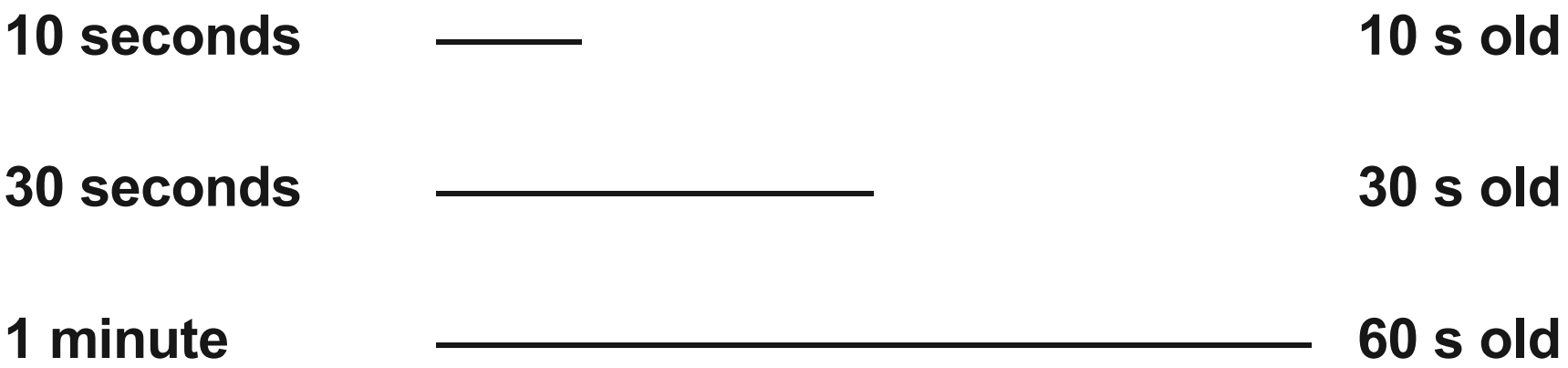


Illustration: age of the snapshot at response arrival, while new commits keep happening.

readyset

READYSET / DEEP CACHE

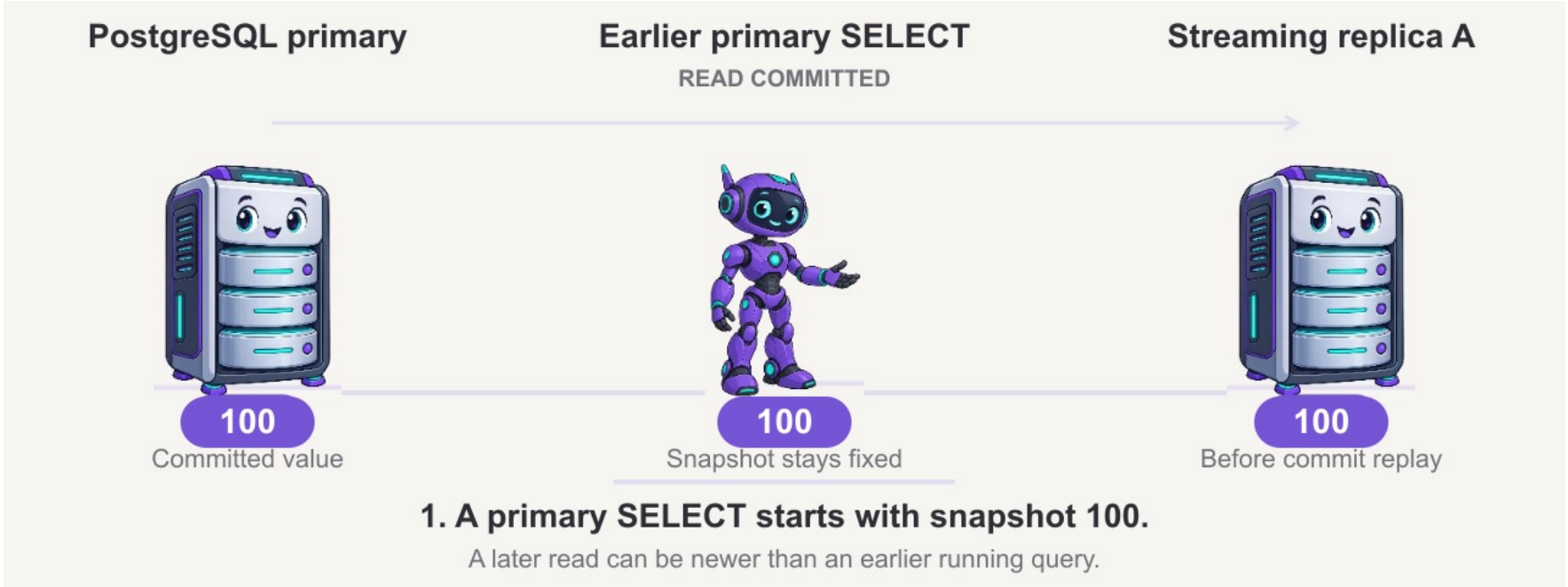
Keep the answer updated.

Read it quickly.

Source changes → maintained result → lookup

readyset

Watch the snapshot fall behind



Local PostgreSQL 18.3 test. Animation timing is illustrative.

readyset

Snapshot age and response time

How old is the visible state when the answer reaches you?

Deep caching can return sooner with newer applied data.

Compare CDC lag + dataflow lag + lookup time with the query’s snapshot age.

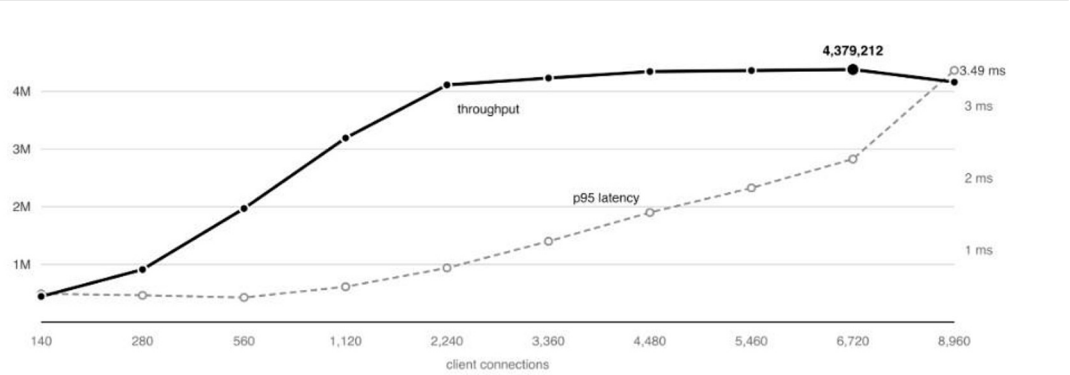
readyset

A published 4.3 million QPS benchmark



4,379,212 queries per second. From a single node.

14 August 2026 · MySQL protocol · fully cached reads · 1 M rows · 10 K hot keys · Table 1



peak throughput

4.38M

queries / second on the isolated cache-hit serving path.

p95 latency at peak

2.26ms

3.49 ms at 8,960 connections, beyond the throughput peak.

where the cycles go

kernel networking **56.3%**

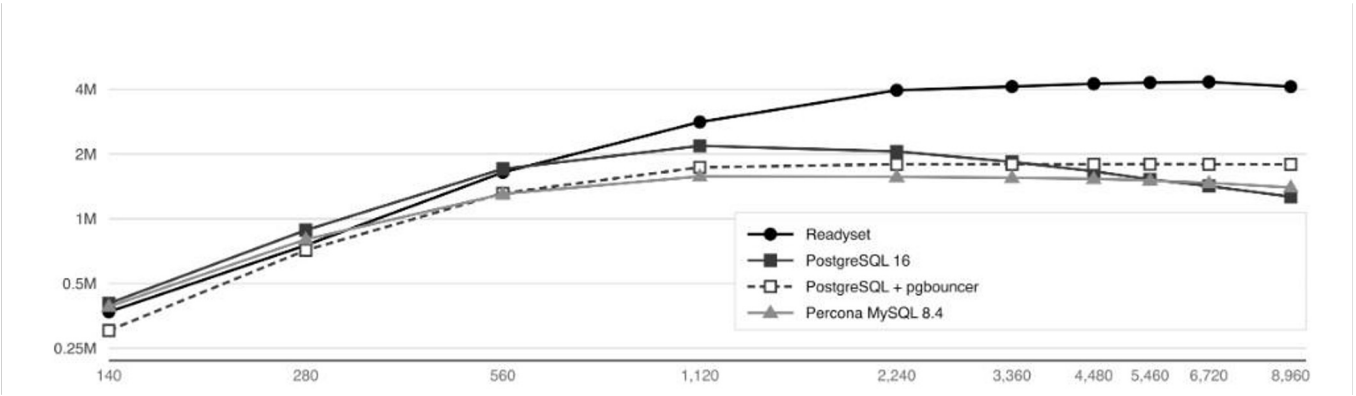
Readyset userspace **42.8%**

cache read path itself **~7%**

Readyset, PostgreSQL, and MySQL

Same server and lookup workload. Protocols differ. This measures cached read throughput, not freshness.

Readyset Engineering · Maximum Query Throughput · Table 3 (separate comparison run)



01 · Readyset

4.33 M peak QPS

Deep cache, query logging off
24 runtime shards
MySQL binary protocol

02 · PostgreSQL 16

2.18 M peak QPS

max_connections: 10,000
shared_buffers: 8 GiB
model: one backend / conn · JIT & parallel off

03 · PostgreSQL + pgbouncer

1.80 M peak QPS

Transaction pool: 6 instances per generator
Prepared statements: 200
Database connections: about 1,050

04 · Percona MySQL 8.4

1.57 M peak QPS

thread handling: pool-of-threads
thread_pool_size: 96
buffer_pool: 8 GiB · conns multiplexed

readyset

Thank you.

Questions?

readyset.io

vinicius@readyset.io